

Regression Models

Applied Machine Learning — Session 2, Chapter 1

Regression: What We're Doing

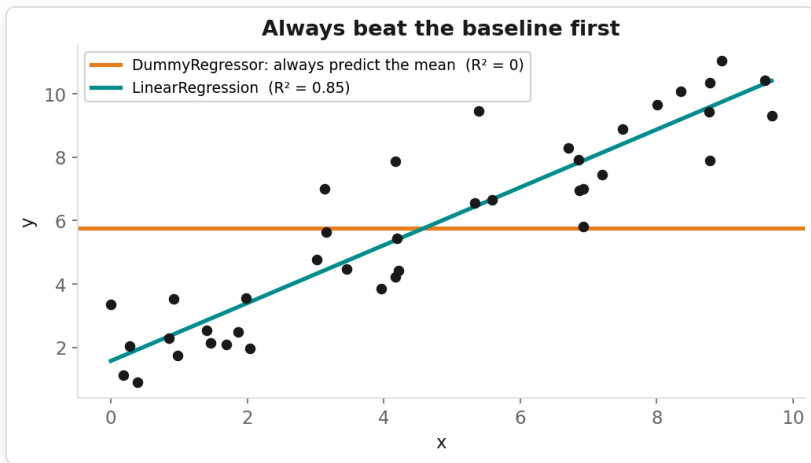
Predicting a continuous number.

- House price given area, location, rooms
- Energy consumption given temperature, day of week
- Patient recovery time given treatment, age, biomarkers

How we measure success (*details in Ch06*):

Metric	Meaning	Units
MAE	average absolute error	same as y
RMSE	like MAE, but big errors count more	same as y
R^2	share of variance explained; 1 = perfect, 0 = "just predict the mean"	none

Baseline First



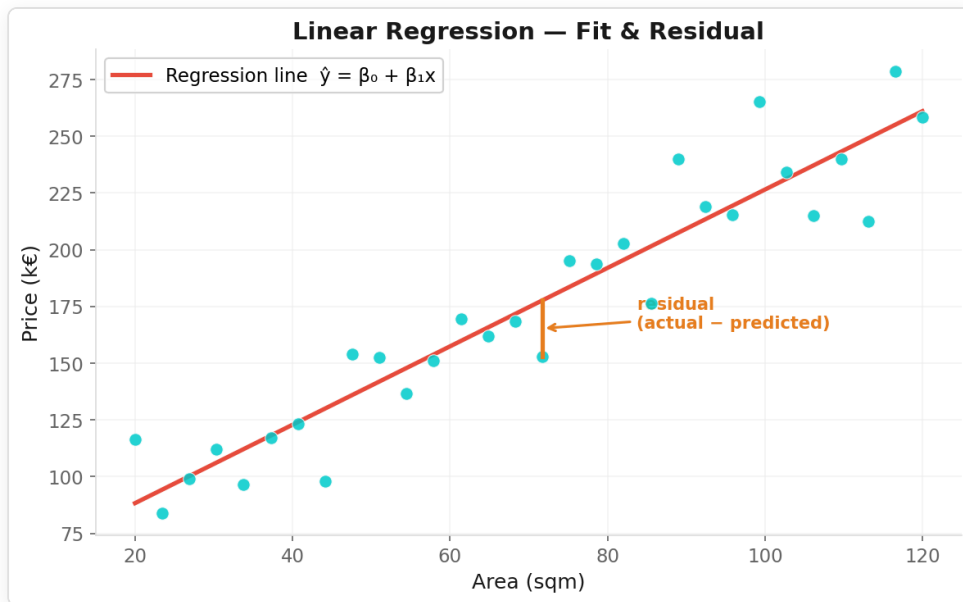
```
from sklearn.dummy import DummyRegressor
DummyRegressor(strategy='mean').fit(X_train, y_train)    #  $R^2 = 0$  by definition
```

Any model that is not clearly better than "predict the average" has learned nothing.

Linear Regression

Model: $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$

Learning: find the β s that make the squared residuals as small as possible (OLS)



Linear Regression in sklearn — and Reading β

```
from sklearn.linear_model import LinearRegression
model = LinearRegression().fit(X_train, y_train)
model.coef_      #  $\beta_1 \dots \beta_n$ 
model.intercept_ #  $\beta_0$ 
```

Reading a coefficient: target in k€, feature `area_sqm` with $\beta = 3.5$

→ each additional m² adds **+3.5 k€ = +3 500 €** to the predicted price, *all other features held constant*

Careful: raw coefficients depend on the units of the feature. To ask *"which feature matters most?"* standardize first (`StandardScaler` in a pipeline) → β = change per **one standard deviation**.

What Does "Best Line" Mean? — The Loss Function

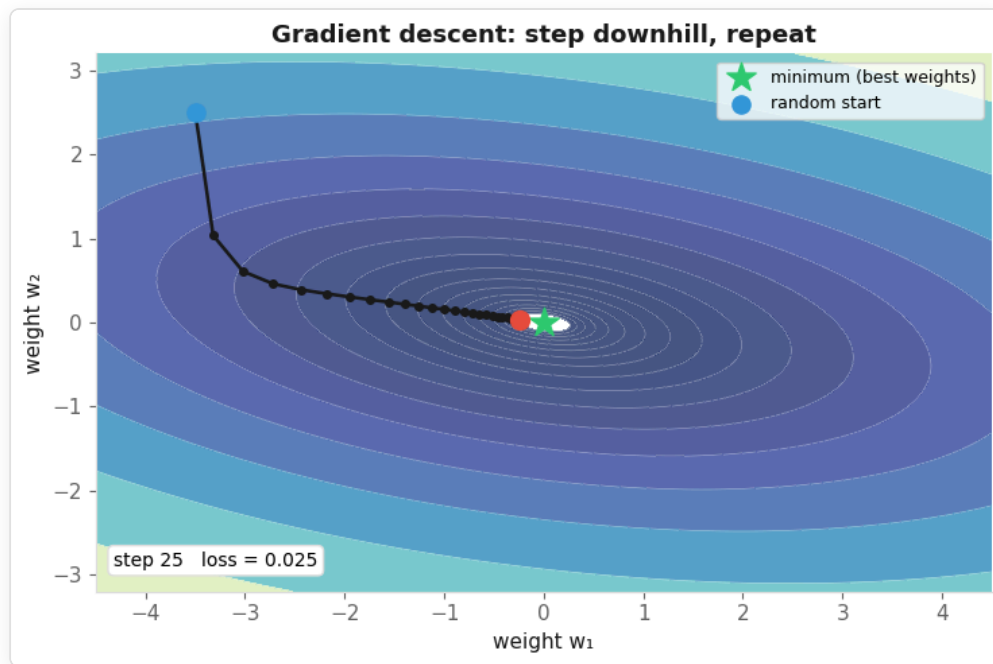
Mean Squared Error — the training loss of linear regression:

$$\text{MSE}(\beta) = (1/n) \sum (y_i - \hat{y}_i)^2 \quad \text{with} \quad \hat{y}_i = \beta_0 + \beta_1 x_{1i} + \dots$$

Training = finding the β that minimizes the loss.

- Linear regression: closed-form solution (one matrix formula) ✓
- Almost everything else (logistic regression, neural nets, boosting): iterative → **gradient descent**

How Do Models Find the Minimum? Gradient Descent



Rule: $w \leftarrow w - \eta \cdot \nabla L(w)$ — step downhill, repeat. η = learning rate: too small = slow, too big = overshoot.

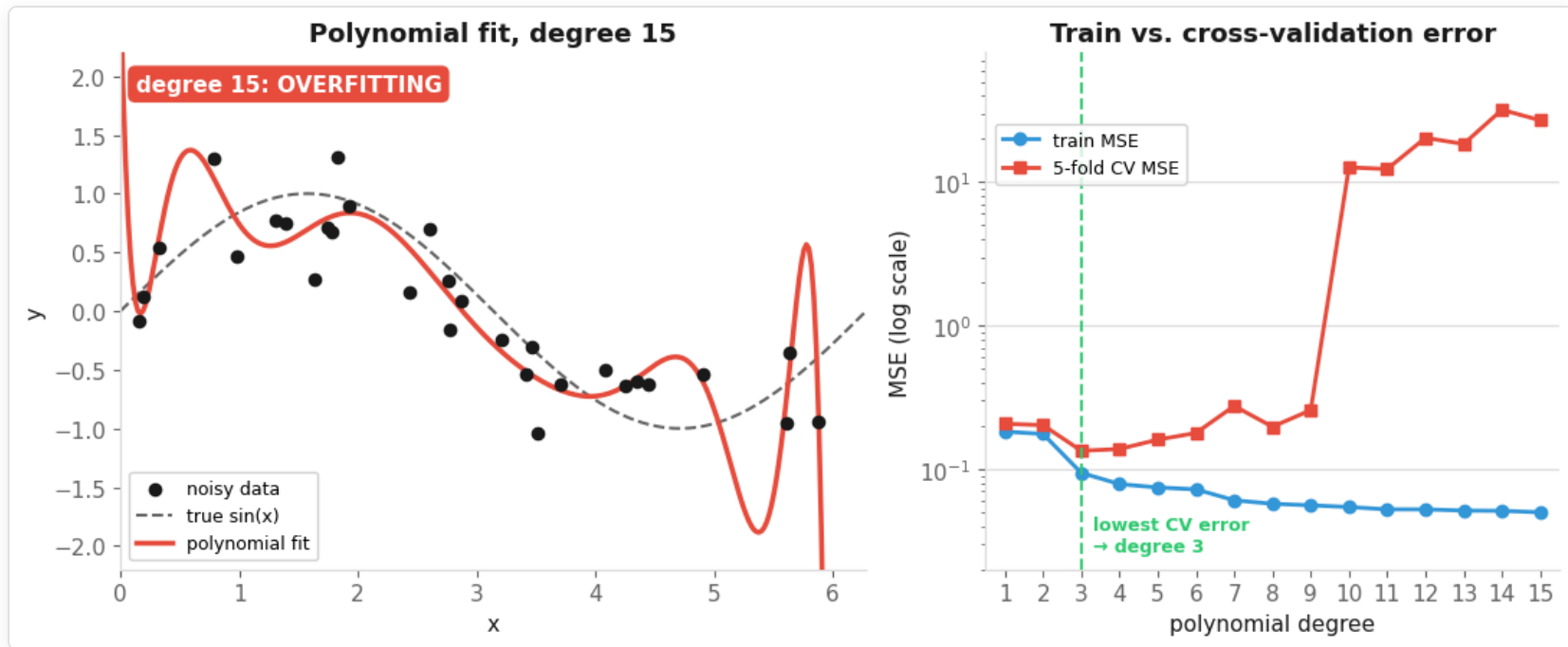
Overfitting Revisited: Polynomial Regression

What if the relationship is curved? Add powers of x , then fit a *linear* model on them.

```
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
model = make_pipeline(StandardScaler(),          # scale x BEFORE powering!
                      PolynomialFeatures(degree=3),
                      LinearRegression())
```

Ch03 showed you degree 1 vs 15 on the sine wave. **Which degree is right — and how do we know?**

Train Error Lies — Cross-Validation Error Tells the Truth



Cross-Validation in Practice

```
from sklearn.model_selection import cross_val_score, KFold

cv = KFold(n_splits=5, shuffle=True, random_state=42)      # shuffle! sorted data → biased folds
scores = cross_val_score(model, X, y, cv=cv, scoring='r2')
print(f'{scores.mean():.3f} ± {scores.std():.3f}')
```

- Every sample is test data **exactly once** → 5 scores instead of 1
- Report **mean ± std** — the std tells you whether a difference between two models is real
- The whole **Pipeline** goes into **cross_val_score** → scaling is re-fit inside every fold (no leakage)

Regularization: Why?

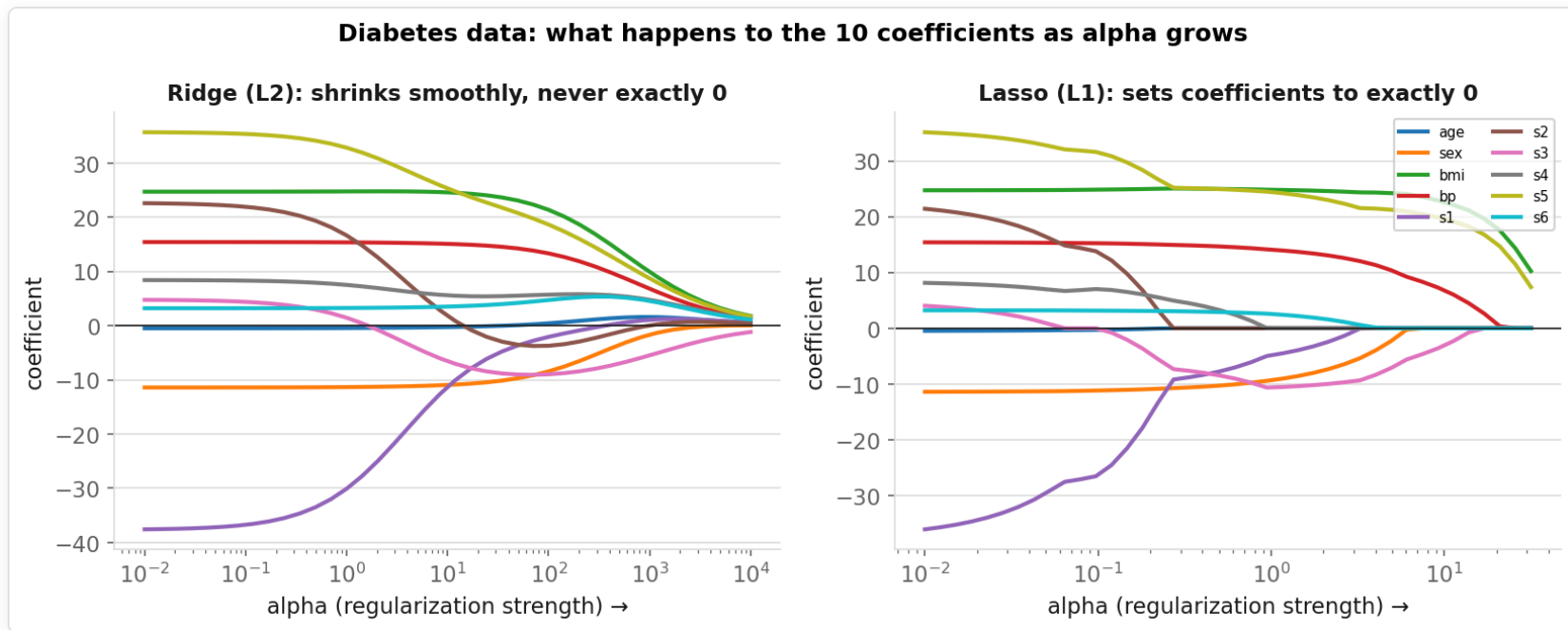
With many (correlated) features and little data, linear models overfit: **huge coefficients that cancel each other out.**

Idea: add a penalty for large coefficients to the loss.

	Ridge (L2)	Lasso (L1)
Loss	$\text{MSE} + \alpha \cdot \sum \beta_i^2$	$\text{MSE} + \alpha \cdot \sum \beta_i $
Effect	shrinks all β smoothly toward 0	sets some β to exactly 0
Use when	all features somewhat useful, correlated	you suspect many irrelevant features

```
from sklearn.linear_model import Ridge, Lasso
Ridge(alpha=1.0)      # alpha = regularization strength (needs scaled features!)
Lasso(alpha=0.1)
```

Watch the Coefficients Shrink



Ridge never reaches zero — **Lasso** kills features one by one → automatic feature selection.

"Always Cross-Validate to Choose α " — Here Is How

```
from sklearn.linear_model import RidgeCV
model = make_pipeline(StandardScaler(),
                      RidgeCV(alphas=np.logspace(-3, 3, 30), cv=5)) # tries 30 alphas x 5 folds
model.fit(X_train, y_train)
model[-1].alpha_ # the winner
```

Same idea for any hyperparameter of any model:

```
from sklearn.model_selection import GridSearchCV
GridSearchCV(RandomForestRegressor(), {'max_depth': [3, 5, 10]}, cv=5).fit(X_train, y_train)
```

Never pick hyperparameters on the test set — the test set is for the final answer only.

Decision Tree Regression

Idea: split the feature space into boxes; predict the **mean of the training targets** in each box.

```
area_sqm > 100 ?  
  yes → rooms > 3 ? → 420 k€ / 280 k€  
  no  → 195 k€
```

```
from sklearn.tree import DecisionTreeRegressor  
DecisionTreeRegressor(max_depth=3, random_state=42)
```

- No feature scaling needed • captures non-linearity and interactions
- ⚠ Deep trees memorize → limit `max_depth` (that is *its* regularization knob)

Random Forest: Wisdom of Crowds

- Train **many** trees, each on a random subset of rows **and** features
- Prediction = **average** of all trees → the individual trees' errors cancel out

```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(n_estimators=100, random_state=42).fit(X_train, y_train)
rf.feature_importances_      # bonus: which features does it use?
```

Almost always better than a single tree — at the cost of interpretability.

Choosing a Regression Model

Model	Interpretable	Non-linear	Needs scaling	Robust to outliers
Linear Regression	✓	✗	✗ (only for comparing β)	✗
Ridge / Lasso	✓	✗	✓	✗
Decision Tree	✓	✓	✗	⚠ (features yes, target no)
Random Forest	⚠	✓	✗	⚠

Rule of thumb: baseline → Linear/Ridge → add a forest if you suspect non-linearity → keep the simplest model that is *good enough*.

Quick Check

Your colleague reports: *"Degree-12 polynomial: train $R^2 = 0.99$. Ship it!"*

1. What single number would you ask for?
2. RidgeCV picks $\alpha = 0.001$ out of $[0.001 \dots 1000]$. What does that tell you?
3. Two models: 0.481 ± 0.085 and 0.478 ± 0.083 (5-fold R^2). Which is better?

Quick Check

Your colleague reports: *"Degree-12 polynomial: train $R^2 = 0.99$. Ship it!"*

1. What single number would you ask for?
 2. RidgeCV picks $\alpha = 0.001$ out of $[0.001 \dots 1000]$. What does that tell you?
 3. Two models: 0.481 ± 0.085 and 0.478 ± 0.083 (5-fold R^2). Which is better?
-
1. **CV (or test) R^2** — train R^2 of a flexible model tells you nothing about new data
 2. Regularization does not help here — the data is plentiful / the model does not overfit (or the grid should extend lower)
 3. **Cannot tell** — the difference (0.003) is far smaller than the fold-to-fold noise (± 0.08)

Now: Exercises!

→ Open `03-exercises/ch04_regression_exercises.ipynb`

Dataset: diabetes progression (442 patients, 10 features) — *not* the housing data → transfer, not copy-paste.

Task	Content
Task 1	baseline + linear regression (3 min)
Task 2	Ridge vs Lasso, which features does Lasso drop? (3 min)
Task 3	random forest + feature importances (4 min)
Bonus	CV comparison · RidgeCV · predicted-vs-actual

A `score()` helper is provided — no metric boilerplate.

Key Takeaways

- **Baseline first** — `DummyRegressor` ; $R^2 = 0$ is the floor
- Linear regression: interpretable — β = change in y per unit (or per std) of x
- Training = minimizing a loss (MSE); gradient descent walks downhill
- **Model complexity** (degree, $1/\alpha$, depth) is a dial: train error always drops, **CV error tells you where to stop**
- Ridge shrinks, Lasso selects; choose α with `RidgeCV` / `GridSearchCV` , never on the test set
- Random Forest: robust non-linear default, feature importances

Next: Chapter 5

Classification Models

"What if the answer is a category, not a number?"